

Gypsum Belt-Filter Cake Recognition

Automated valid / invalid detection from a plant camera feed

AI Expert — Final Project

Yaron Conroy

2026-09-24

Code: github.com/yaronconroy-del/picture-recognition-gypsum

Problem Statement

- A fixed camera watches a gypsum belt filter on a phosphoric-acid production line.
- Filtration quality is currently checked by manual visual inspection — intermittent, and inconsistent between operators/shifts.
- Goal: an image classifier that looks at a single camera frame and tells whether the cake is VALID (properly dewatered) or INVALID (wet gypsum), automatically and continuously.
- This is a proxy for a real yield-loss and equipment-damage problem, not a cosmetic visual check (see next slides).

Background: the Process

- Wet-process phosphoric acid production:
 - Phosphate rock + sulfuric acid → phosphoric acid + gypsum (by-product)
- Gypsum still carries residual phosphoric acid.
- It's washed with water, then dewatered on the belt filter this project's camera watches — to recover as much phosphoric acid as possible before the gypsum is discarded.
- “Invalid” = wet gypsum: the cake didn't dewater properly.

Rock + H₂SO₄

↓

Phosphoric acid + Gypsum (wet)

↓ wash with water

↓ belt filter (dewater)

Dry cake → recovered acid | Wet cake → lost acid

Background: the Cost of Bad Filtration

- Wet gypsum causes two separate costs:
 - Yield loss — unrecovered phosphoric acid, lost with the discarded gypsum: ~\$2,000/hour per 1% P₂O₅ remaining
 - Equipment damage — wet gypsum can damage downstream equipment, adding downtime

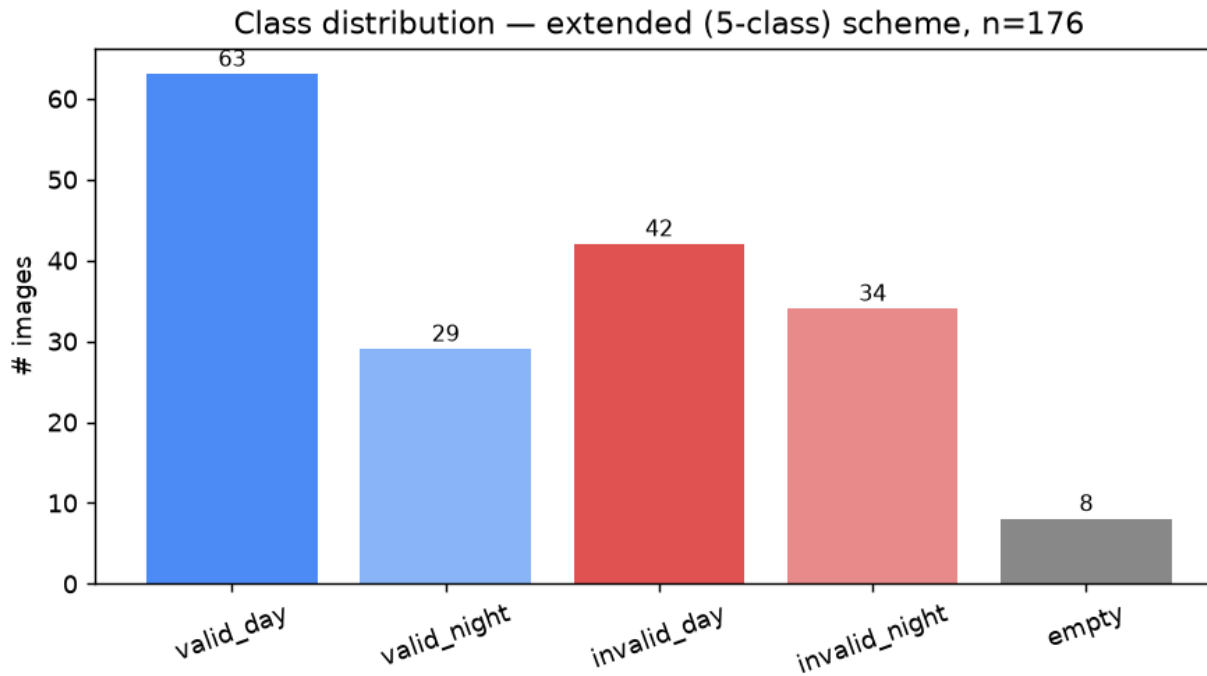
Yield-loss rate	\$2,000/hr per 1% P ₂ O ₅ → ~\$6,000/hr at a typical ~3% equivalent
Yield-loss exposure	~480 hr/yr affected (6% of ~8,000 hr/yr) → ~\$2,880,000/year
Downtime rate (equipment damage)	\$49,500/hr (45 × 1,100)
Downtime exposure	~240 hr/yr affected (3% of ~8,000 hr/yr) → ~\$11,880,000/year
Combined annual exposure	~\$14,760,000/year
Automated-detection recovery (10–15%)	~\$1,476,000 – \$2,214,000/year

Assumptions stated explicitly — swap in real annual operating hours for exact figures.

Related Work

- Transfer learning from ImageNet-pretrained CNNs is standard for small industrial inspection datasets — used in 83% of studies in a 2024 review of CNN-based surface-defect detection.
 - This project follows that pattern: MobileNetV2, transfer-learned, not trained from scratch.
- Class imbalance between “normal” and “defect” is treated as the norm in this literature — defects are genuinely rarer than normal output.
 - This project's “empty” class (8 of 176 images, 4.5%) sits in the same range; class-weighted loss is the standard mitigation, which this project uses.
- Published benchmarks reach 95%+ accuracy — but on datasets one to three orders of magnitude larger than this project's 176 images.
 - This project's ~80% CV accuracy should be read against that gap, not against best-case published numbers.

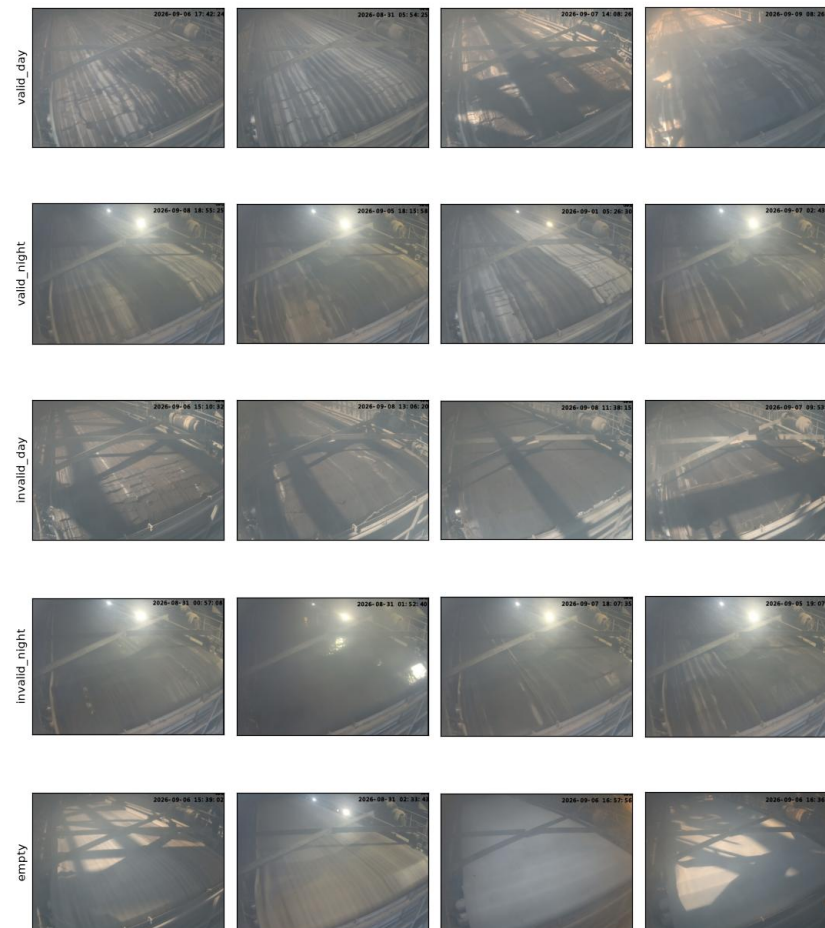
Dataset



- 176 photos total, from the plant's own camera (via WhatsApp export).
- 5 native classes: valid/invalid × day/night, plus empty filter.
- Counts shown after the label correction — one photo moved from valid_night to valid_day after a manual review (see the data-quality slide).
- Randomized filenames + a manifest for traceability; real capture-session timestamps recovered from it for leakage-safe splitting.
- IT/data-access constraints limited the photos available for this phase — a full-scale pilot would use a much larger database.

Dataset — What the Camera Actually Sees

Sample images per class (random, seed=42)



Preprocessing & Modeling Approach

- Timestamp overlay masked out (top strip) so the model can't key off the burned-in clock.
- Leakage-safe splits by real capture session (not per-image) — photos seconds apart don't leak across train/test.
- Three split ratios tested (80/20, 75/25, 70/30) plus K-fold cross-validation, auto-capped per scheme to what the scarcest class's session count actually supports.
- Model: MobileNetV2 (52 conv layers, ImageNet-pretrained) — transfer learning, not training from scratch (§1.1).
 - Only the last block + new classifier are unfrozen; the rest stays at pretrained weights — 176 images can't support fully fine-tuning 3.4M parameters without overfitting. A larger database would support unfreezing more (§16.3).
- Class-weighted CrossEntropyLoss for imbalance; a WeightedRandomSampler variant was also tried.
- One fixed config (backbone, lr=1e-3, 10 epochs) held constant across every comparison, so only one variable changes at a time.

Experiment 1: Which Label Scheme?

Cross-validation accuracy (the trustworthy number — averaged over independent sessions):

Scheme	CV Accuracy	Invalid Recall
Lite (3-class: valid/invalid/empty)	63.3% $\pm$ 2.7	43.3% $\pm$ 12.4
Extended (5-class: + day/night)	67.6% $\pm$ 4.9	49.9% $\pm$ 14.4
No-empty collapsed (2-class)	70.0% $\pm$ 14.7	56.1% $\pm$ 25.5 (unstable)
No-empty split (4-class)	80.3% $\pm$ 0.4	80.7% $\pm$ 1.6

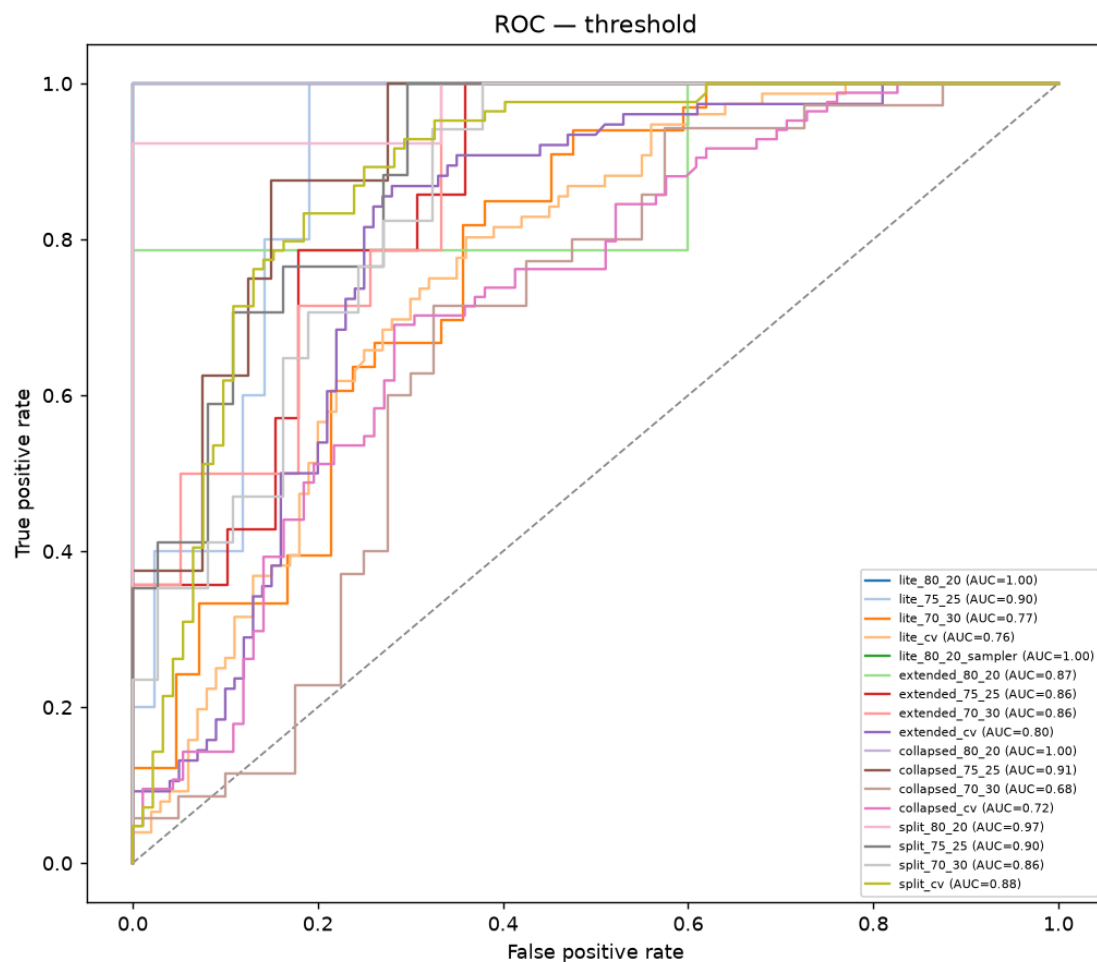
- No-empty split (4-class) beat every scheme tried — and with the tightest spread (± 0.4 points): folding “empty” into “invalid” measurably helps, not just simplifies.
- Extended (5-class) beat Lite in CV — keeping day/night as classes carries real signal.
- No-empty collapsed's high single-split numbers are not trustworthy — its CV variance is enormous (± 14.7 points).

Decision: No-Empty Split (4-class)

- Compared fairly (same ground-truth definition, same run): no-empty split wins on every metric vs. extended — CV accuracy (80.3% vs. 67.6%), ROC AUC (0.882 vs. 0.782), PR AP (0.826 vs. 0.704).
- Beyond the numbers — the operational argument is stronger:
 - The empty-filter dataset is tiny (8 of 176 images).
 - A live model doesn't need to detect “empty” at all — the operator empties the filter themselves and already knows it.
 - Production design: toggle the model off during that operator-controlled window, back on once filtration resumes.

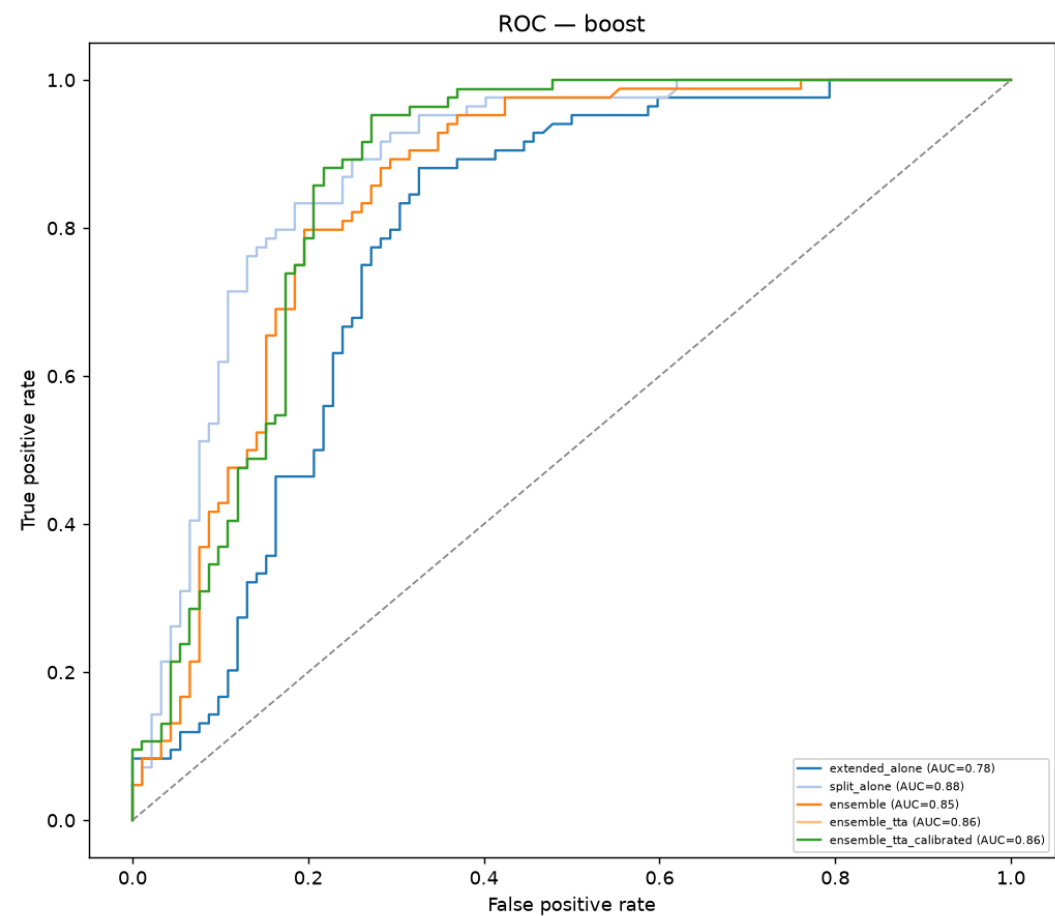
Classes: valid_day · valid_night · invalid_day · invalid_night

Experiment 2: Tuning the Decision Threshold



- A bare `argmax()` decides valid-vs-invalid with an untuned, implicit 0.5 cutoff.
- Every config re-analyzed with full probabilities: ROC/AUC, precision-recall/AP, calibration, and two threshold candidates (Youden's J; F1-optimal).
- Finding: the default 0.5 cutoff under-flags real faults — 17 of 17 configs have BOTH optimal thresholds below 0.5.
- No-empty split CV (promoted): F1-optimal 0.37, Youden 0.49 → operate at ~0.37–0.49.
- Extended CV: F1-optimal 0.30 — well below 0.5.

Experiment 3: Pushing Further (a Negative Result)



Variant	ROC AUC	PR AP
Extended alone	0.782	0.704
No-empty split alone	0.882	0.826
Ensemble (avg. of both)	0.846	0.774
+ Test-time augmentation	0.862	0.795
+ Calibration	0.862	0.795

- Tried: ensembling extended + no-empty split, test-time augmentation, Platt-scaling calibration.
- None beat the single best model — reported straight, not spun.
- A plain average pulled the stronger model down toward the weaker one; TTA recovered only part of it.

Finding: A Mislabeled Photo Was Costing Accuracy

- All 168 day/night photos were reviewed by hand at full resolution against the one reliable visual cue: night's sharp floodlight glare vs. day's flatter, diffuse haze.
- One photo was filed wrong: a “valid, night” image with no floodlight glare anywhere — moved to “valid, day”. Every result in this deck is retrained on the corrected data.

Promoted model (no-empty split, CV)	Before fix	After fix
Accuracy	77.4% ± 1.9	80.3% ± 0.4
Invalid recall	71.1% ± 5.4	80.7% ± 1.6
ROC AUC	0.818	0.882
PR AP	0.768	0.826
Control: lite CV accuracy (unaffected by day/night)	63.3%	63.3%
Control: collapsed CV accuracy (unaffected)	70.8%	70.0%

- The schemes the fix doesn't touch reproduce almost exactly, so the gain comes from the fix, not run-to-run noise. Extended (also day/night-based) dropped to 67.6% — reported as-is; the decision stands.

Live Demo Scripts

- `predict.py` — classifies one photo from the command line: predicted class, confidence, and a VALID/INVALID call.
 - Runs the promoted model from the final run (no-empty split, 70/30) automatically; “empty” predictions count as INVALID, matching the §12 decision.
 - Spot-checked: an “invalid, day” photo → INVALID (61% confidence). Earlier 3-photo check on the runner-up model: 2/3 correct, the miss on the weak “empty” class.
- `watch_camera_feed.py` — simulates the live feed: watches a folder, classifies each new photo within seconds, prints an ALERT line for INVALID.
 - A stand-in for a real camera connection — tested locally with a valid and an invalid sample photo, both classified and flagged correctly.
- Both reuse the exact same preprocessing as training — no drift between what was evaluated and what runs live.

Conclusions

- No-empty split (4-class) is the strongest model found — $80.3\% \pm 0.4$ CV accuracy, 80.7% invalid recall, ROC AUC 0.882.
- Folding “empty” into “invalid” is both statistically and operationally correct, not just a simplification.
- Data quality mattered: fixing one mislabeled photo lifted the promoted model's CV accuracy from 77.4% to 80.3% and invalid recall from 71.1% to 80.7%.
- The default 0.5 threshold under-flags real faults — tuned thresholds catch more real invalid cases.
- Ensembling/TTA/calibration did not beat the single best model — an honest negative result.
- Business case: automated detection is estimated to recover ~\$1.48M–2.21M/year (10–15% of ~\$14.76M/year combined yield-loss + downtime exposure).

Limitations

- Small dataset by IT constraint, not choice — 176 images (8 “empty”); every number carries real uncertainty.
- The final run is a single seed on CPU — variance across seeds not yet measured beyond the CV fold spread.
- Day/night still isn't explained — not literal time-of-day, not whole-image brightness — though the manual review found the labels themselves reliable (1 error in 168).
- The live-feed demo is a simulation (folder-watcher), not a real camera/RTSP integration or real alerting channel.
- Single fixed camera, uncontrolled lighting; source images are WhatsApp-recompressed, not native resolution.

Further Directions

Near-term

- Repeat the final run across several seeds to measure run-to-run variance.
- Confidence-weighted ensembling instead of a plain average; build the real camera feed + alerting channel.

Full-scale pilot (plant engineer's roadmap)

- Retrain on a much larger database once IT/data-access constraints are lifted, and unfreeze more of the backbone (today's last-block-only fine-tuning is a dataset-size-driven choice, not a ceiling).
- Add projectors around the filter to normalize day/night lighting and reduce shadow effects — engineers the day/night ambiguity away at the source.
- Move beyond binary valid/invalid to a continuous filtration-quality score, so the process trend is visible.
- Make it a time-based model that tracks change over a sequence of frames and flags the process turning bad — early warning, not single-frame classification.
- Far future: a downstream model that calculates the right reactor process parameters from the predicted filtration state — closing the loop from vision to process control.

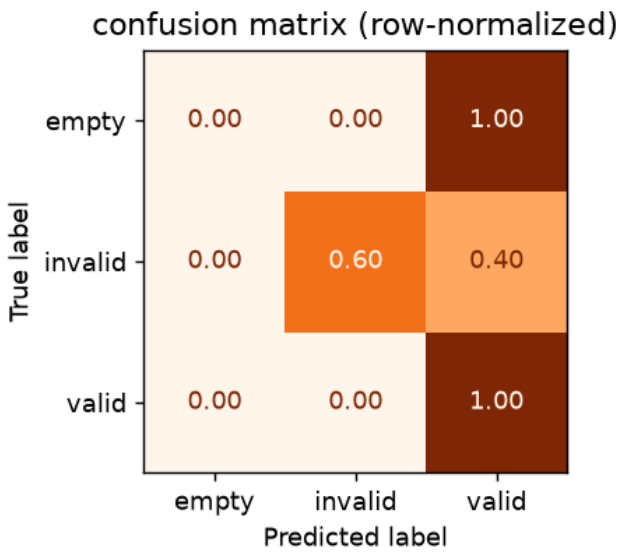
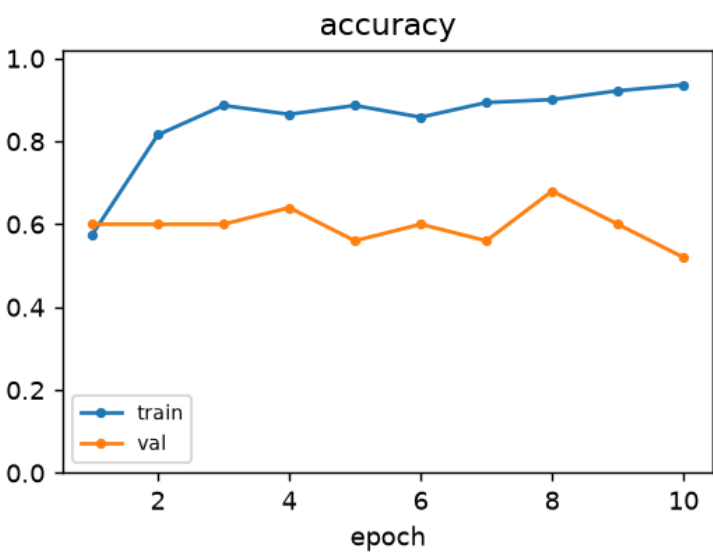
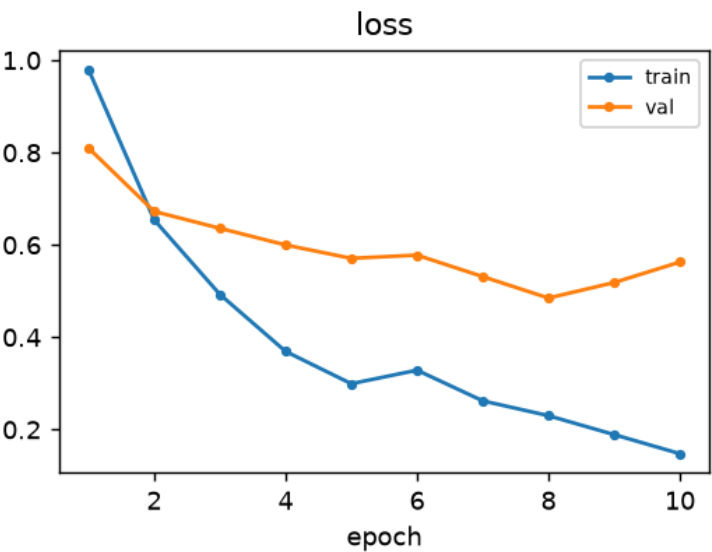
Appendix: Every Model

Training curves (loss and accuracy, train vs. val) and the confusion matrix for all 17 configurations of the final run.
CV configurations show the mean curve across folds and the confusion matrix summed over folds.

Lite (3-class), 80/20 split

accuracy 60.0% · invalid recall 60.0% · macro F1 0.450 · ROC AUC 1.000 · F1-optimal threshold 0.27 (10 test images)

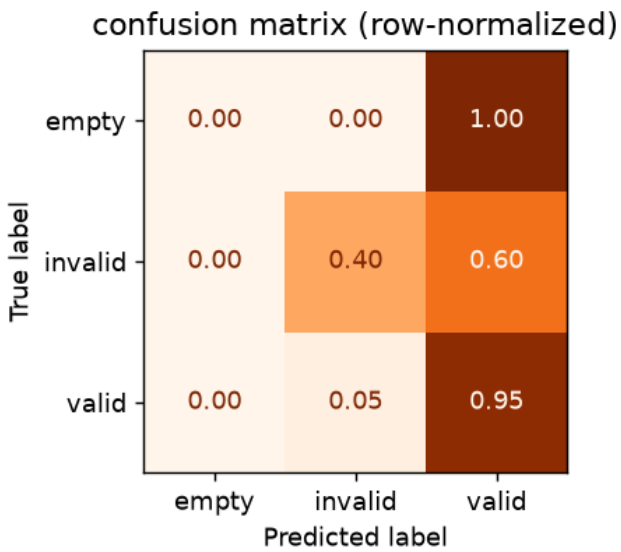
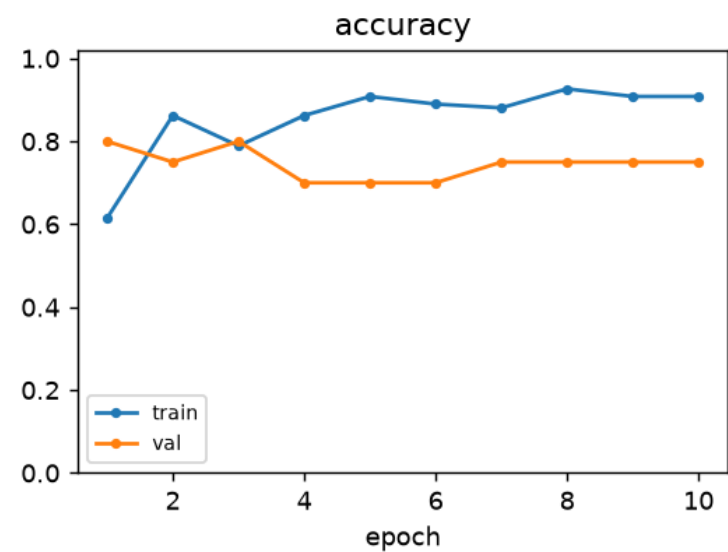
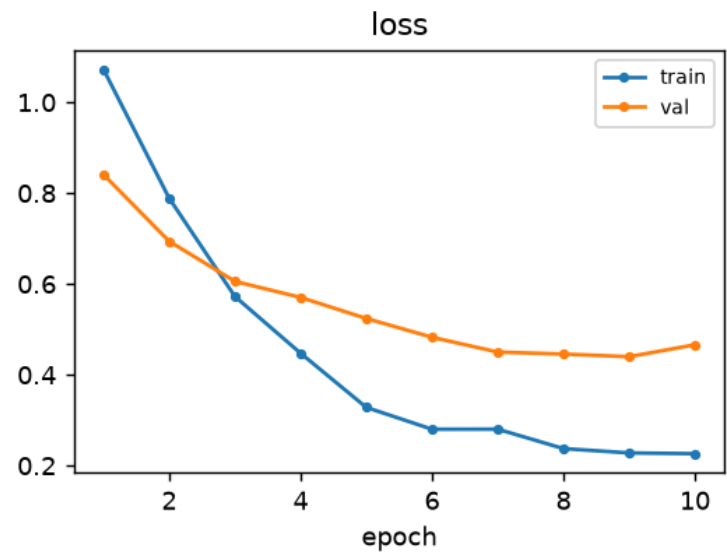
lite_80_20 — ratio split



Lite (3-class), 75/25 split

accuracy 85.1% · invalid recall 40.0% · macro F1 0.453 · ROC AUC 0.905 · F1-optimal threshold 0.22 (47 test images)

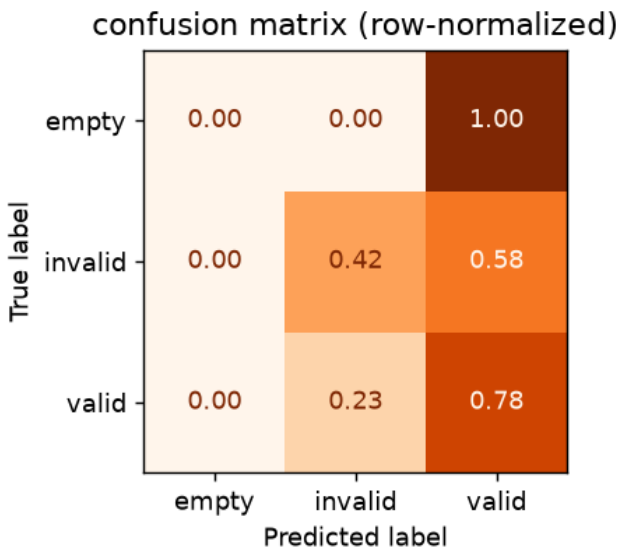
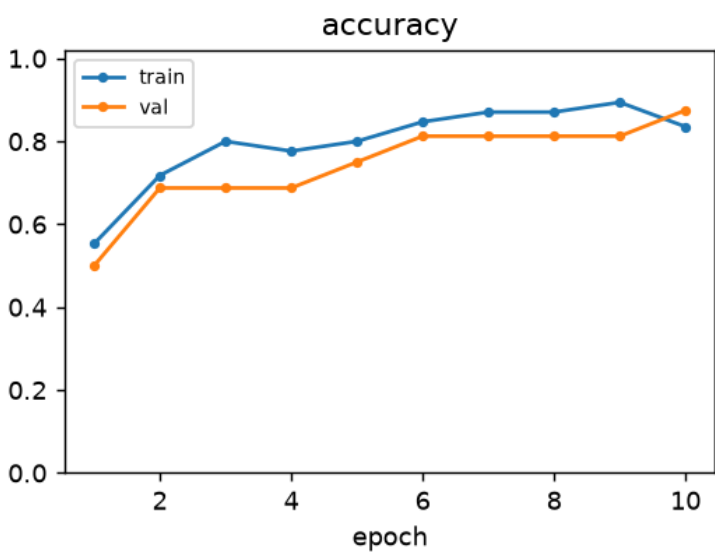
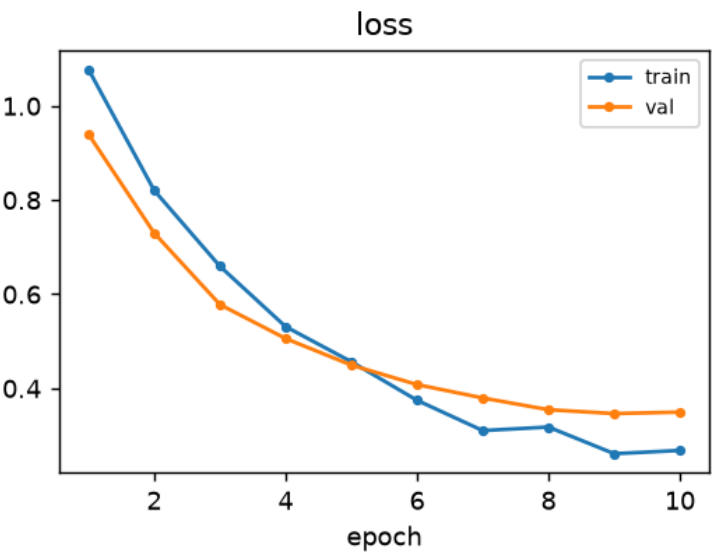
lite_75_25 — ratio split



Lite (3-class), 70/30 split

accuracy 60.0% · invalid recall 42.4% · macro F1 0.391 · ROC AUC 0.773 · F1-optimal threshold 0.15 (75 test images)

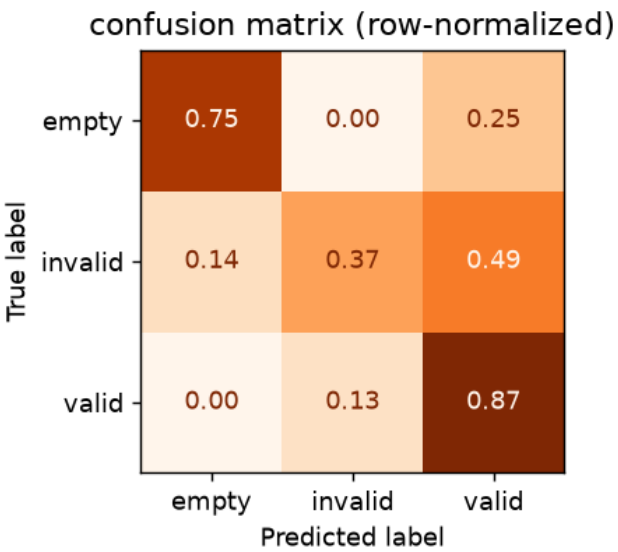
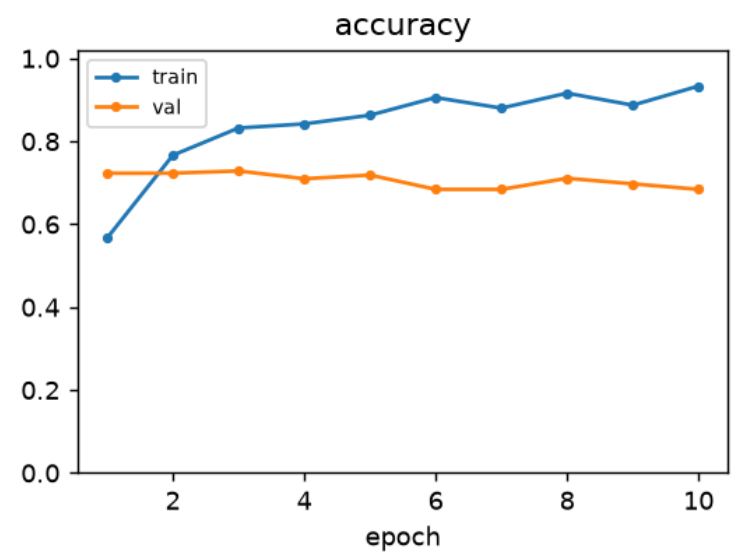
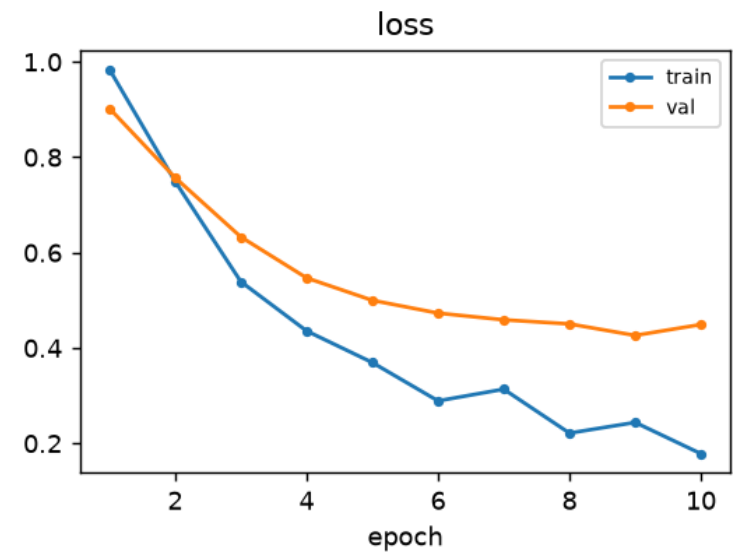
lite_70_30 — ratio split



Lite (3-class), 3-fold cross-validation

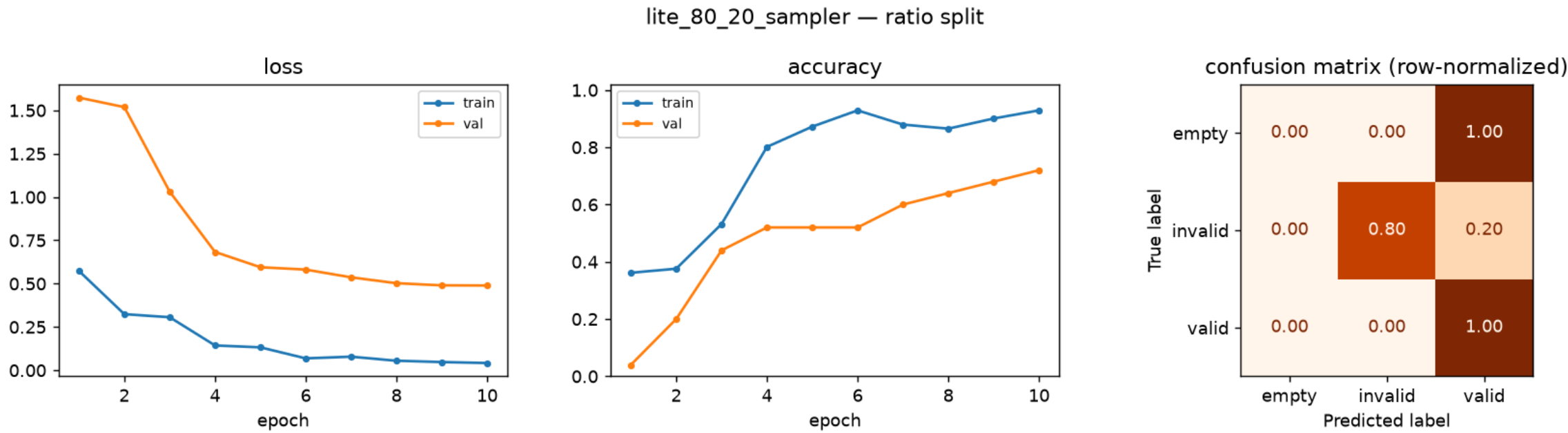
accuracy 63.3% ± 2.7 · invalid recall 43.3% ± 12.4 · macro F1 0.540 · ROC AUC 0.763 · F1-optimal threshold 0.11 (176 pooled test images)

lite_cv — mean over 3 CV folds



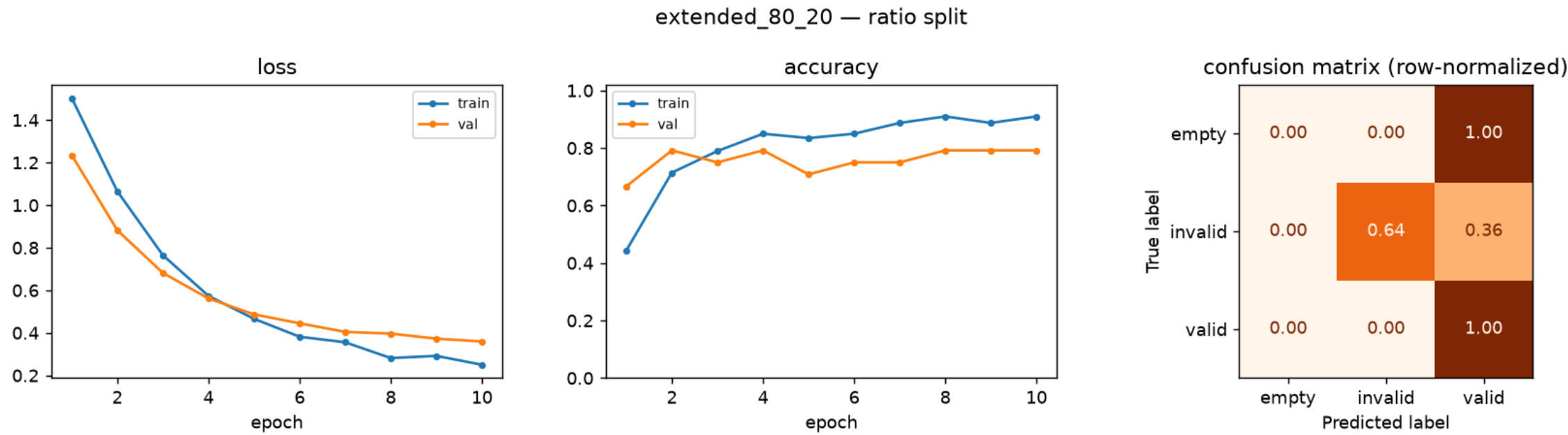
Lite (3-class), 80/20 split + weighted sampler

accuracy 70.0% · invalid recall 80.0% · macro F1 0.519 · ROC AUC 1.000 · F1-optimal threshold 0.40 (10 test images)



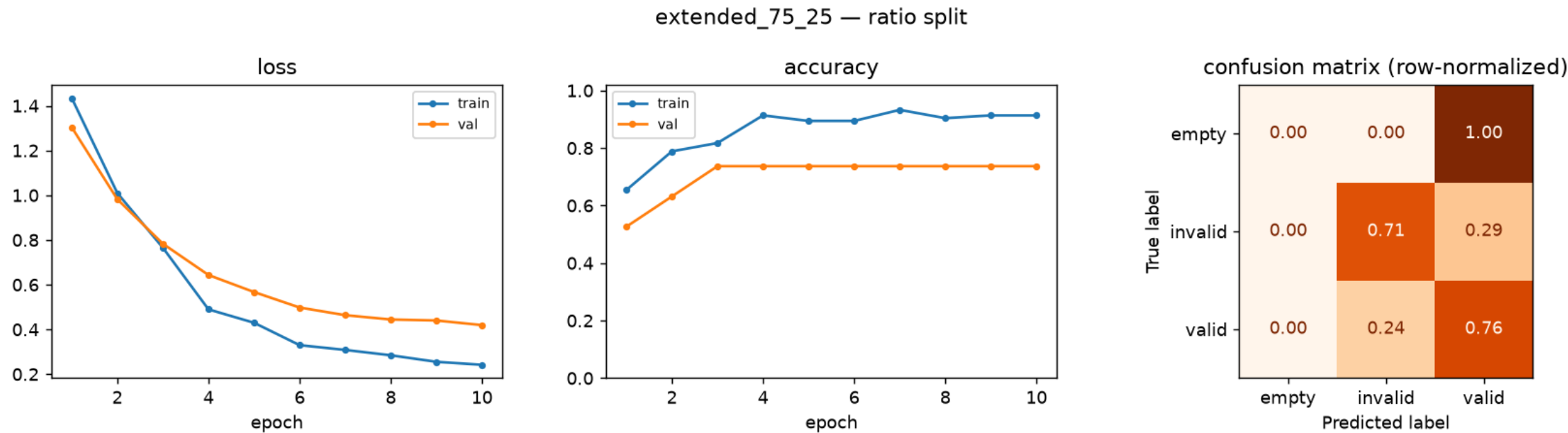
Extended (5-class), 80/20 split

accuracy 63.2% · invalid recall 64.3% · macro F1 0.415 · ROC AUC 0.871 · F1-optimal threshold 0.14 (19 test images)



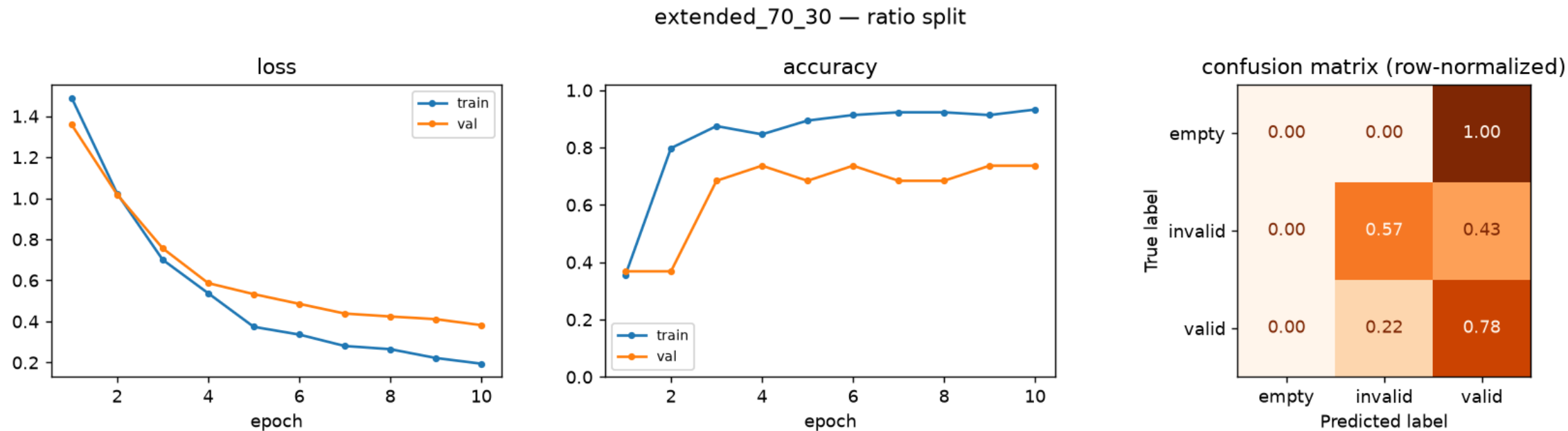
Extended (5-class), 75/25 split

accuracy 71.7% · invalid recall 71.4% · macro F1 0.465 · ROC AUC 0.859 · F1-optimal threshold 0.48 (53 test images)



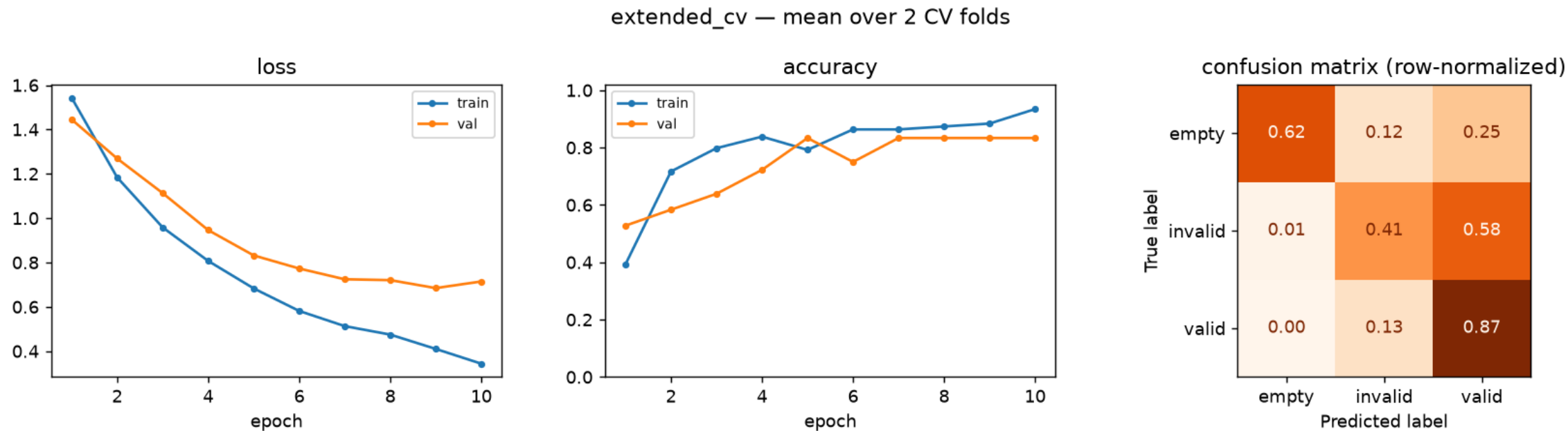
Extended (5-class), 70/30 split — runner-up model

accuracy 69.8% · invalid recall 57.1% · macro F1 0.439 · ROC AUC 0.864 · F1-optimal threshold 0.22 (53 test images)



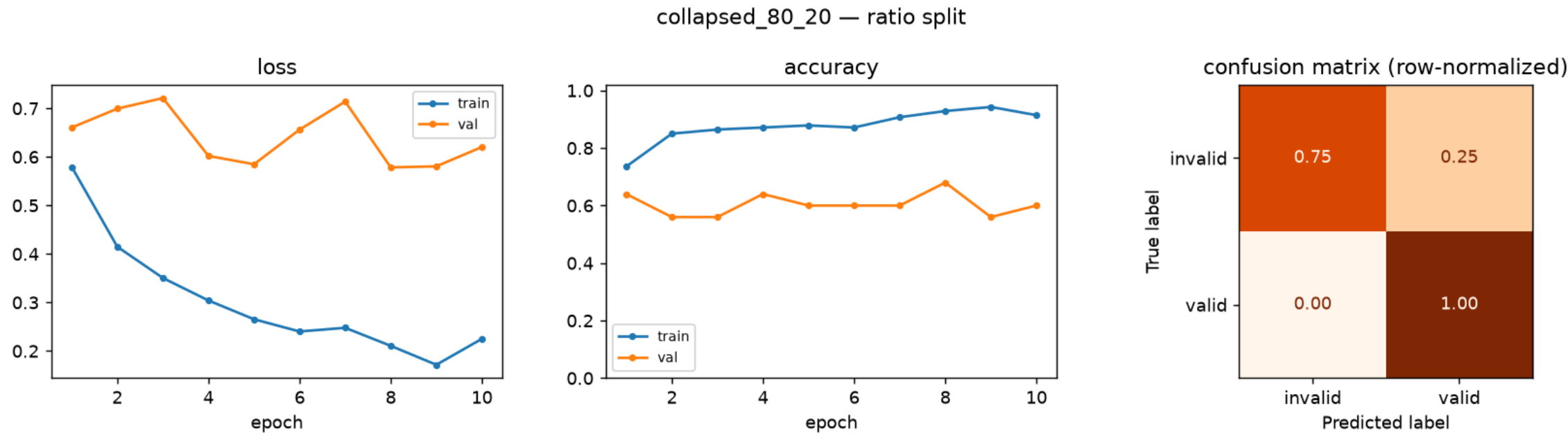
Extended (5-class), 2-fold cross-validation

accuracy 67.6% ± 4.9 · invalid recall 49.9% ± 14.4 · macro F1 0.642 · ROC AUC 0.797 · F1-optimal threshold 0.30 (176 pooled test images)



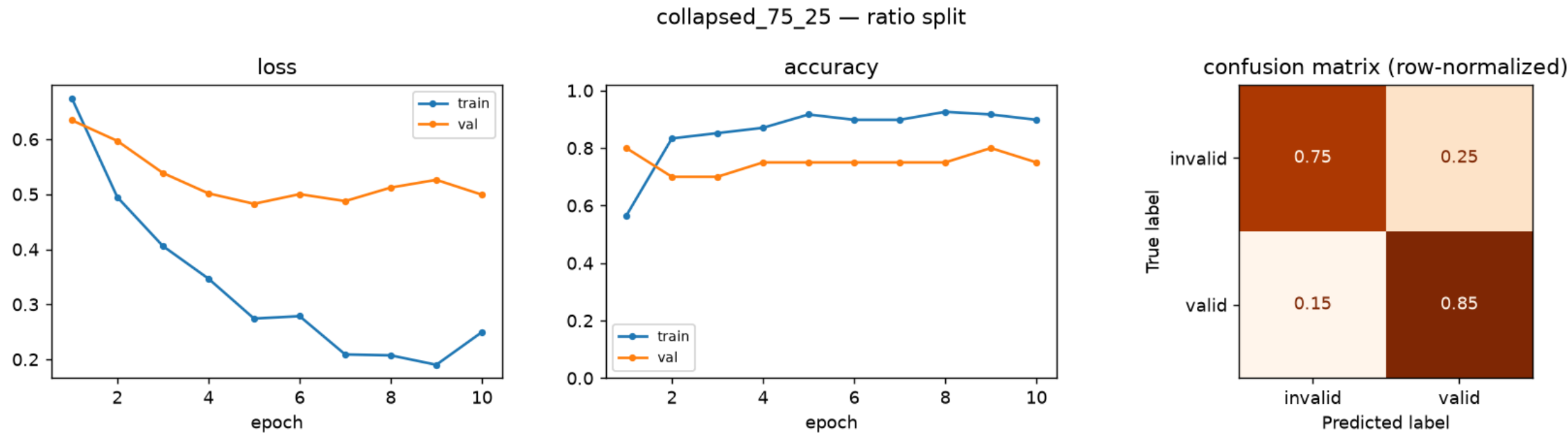
No-empty collapsed (2-class), 80/20 split

accuracy 81.8% · invalid recall 75.0% · macro F1 0.804 · ROC AUC 1.000 · F1-optimal threshold 0.17 (11 test images)



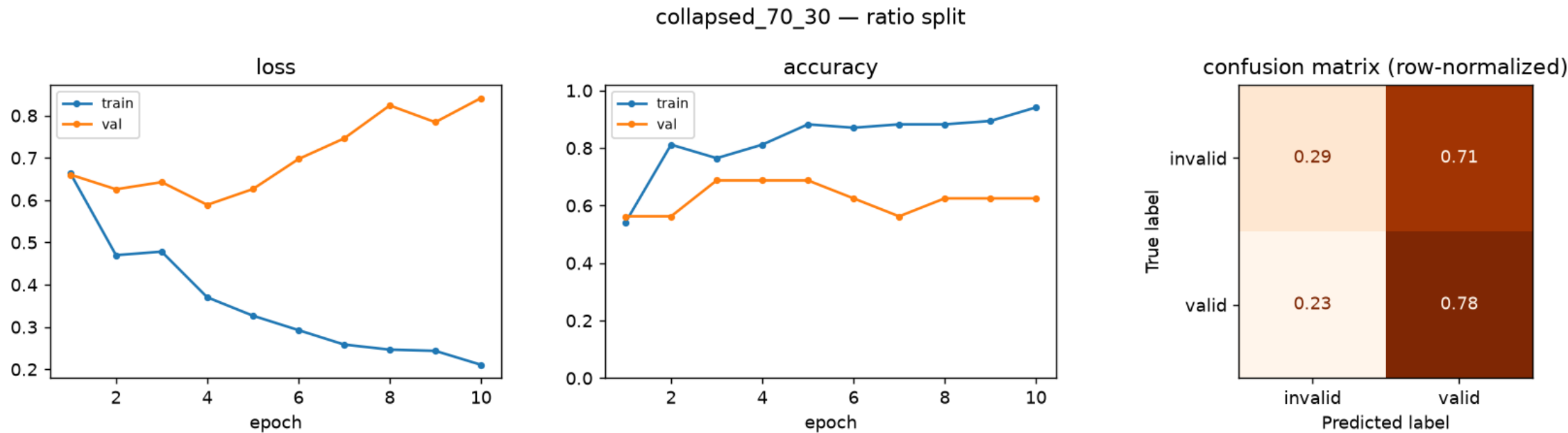
No-empty collapsed (2-class), 75/25 split

accuracy 83.3% · invalid recall 75.0% · macro F1 0.747 · ROC AUC 0.912 · F1-optimal threshold 0.44 (48 test images)



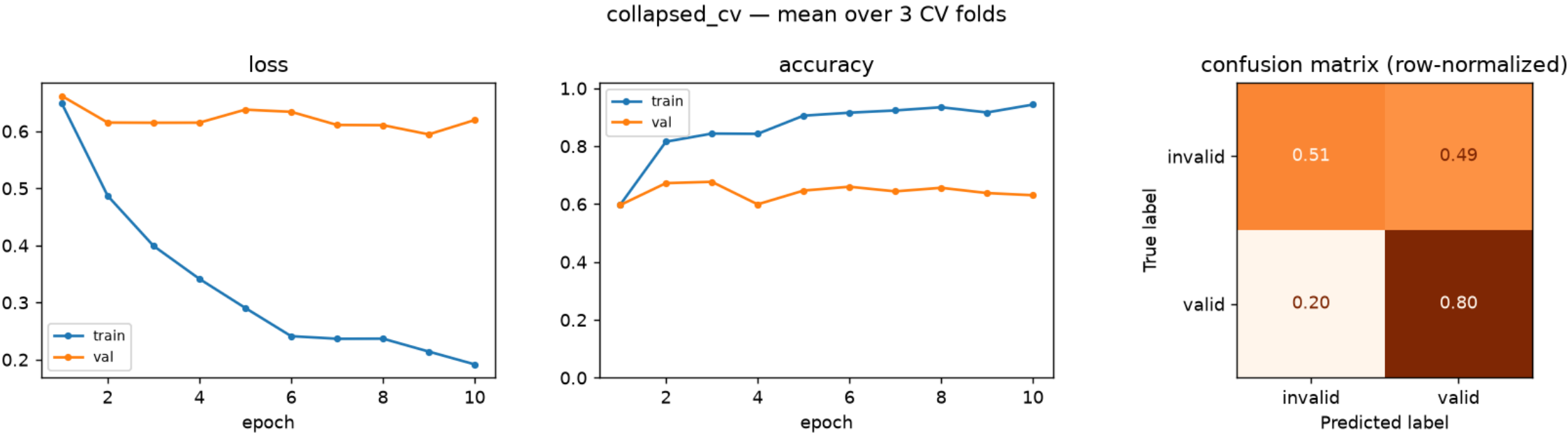
No-empty collapsed (2-class), 70/30 split

accuracy 54.7% · invalid recall 28.6% · macro F1 0.508 · ROC AUC 0.681 · F1-optimal threshold 0.12 (75 test images)



No-empty collapsed (2-class), 3-fold cross-validation

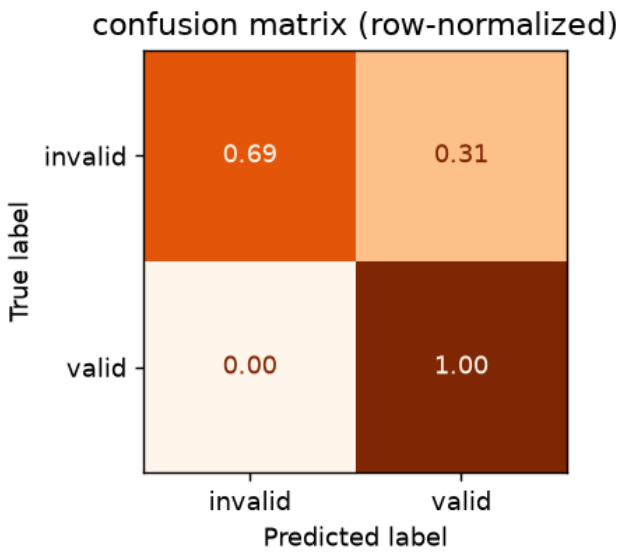
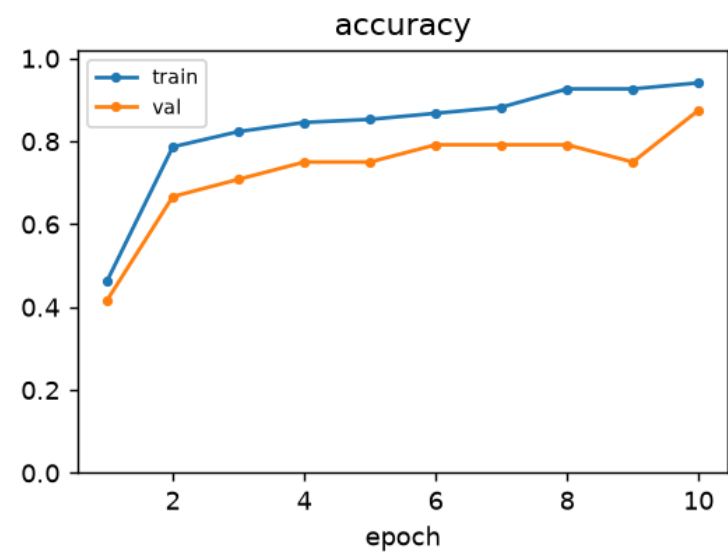
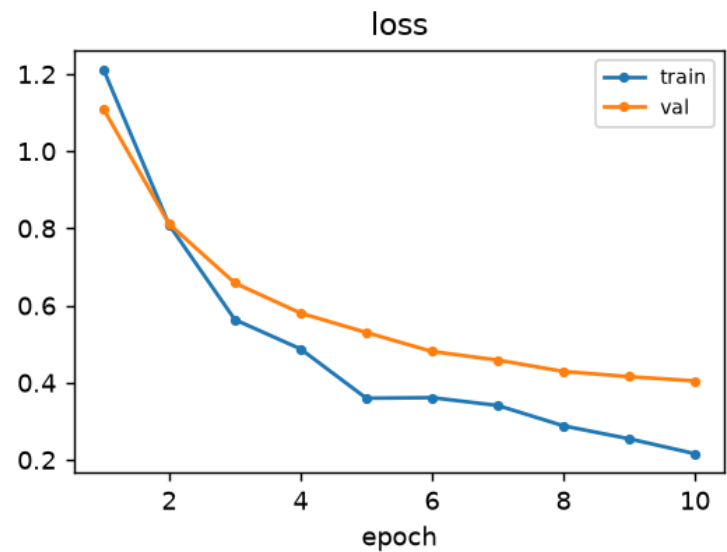
accuracy 70.0% ± 14.7 · invalid recall 56.1% ± 25.5 · macro F1 0.677 · ROC AUC 0.723 · F1-optimal threshold 0.17 (176 pooled test images)



No-empty split (4-class), 80/20 split

accuracy 75.0% · invalid recall 69.2% · macro F1 0.709 · ROC AUC 0.974 · F1-optimal threshold 0.23 (16 test images)

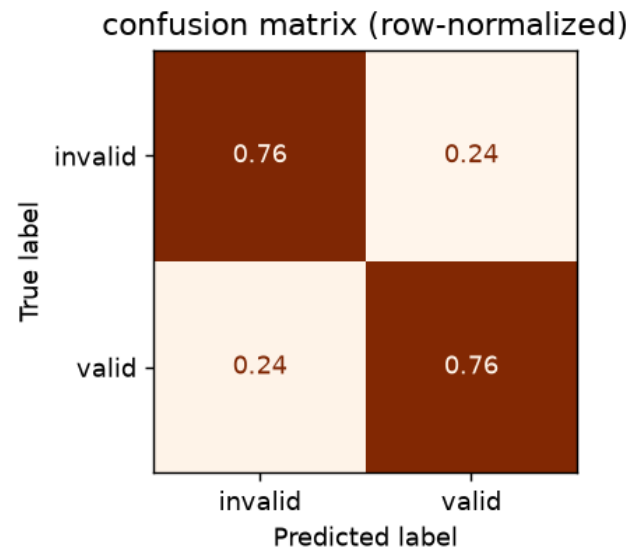
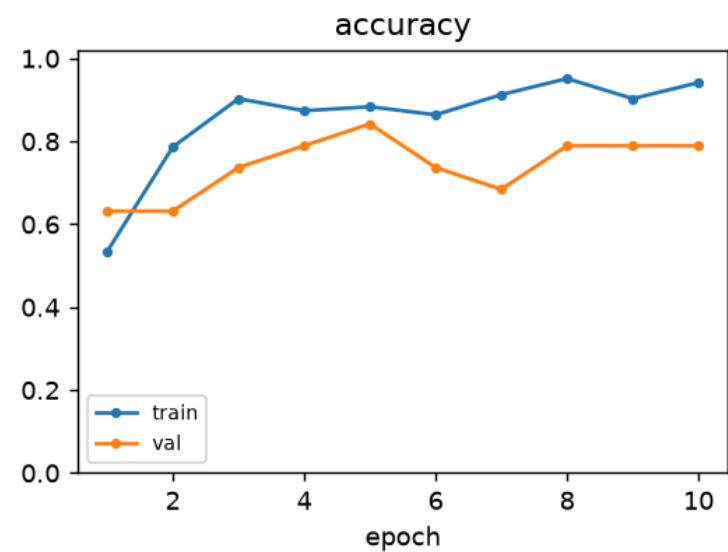
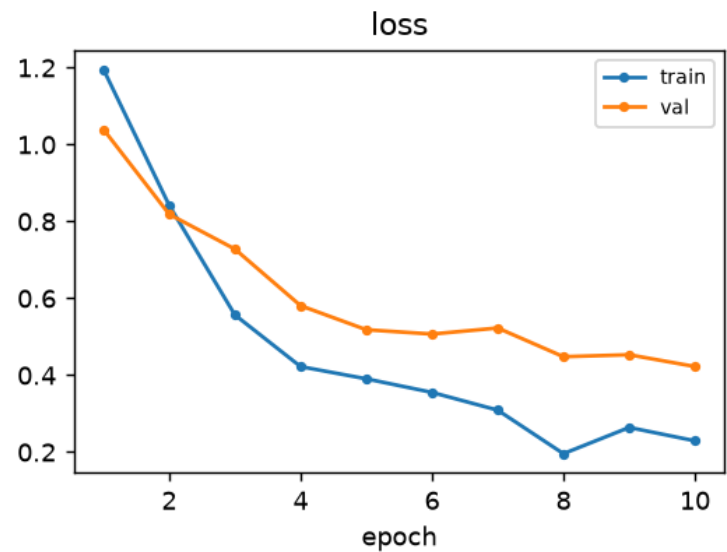
split_80_20 — ratio split



No-empty split (4-class), 75/25 split

accuracy 75.9% · invalid recall 76.5% · macro F1 0.739 · ROC AUC 0.895 · F1-optimal threshold 0.35 (54 test images)

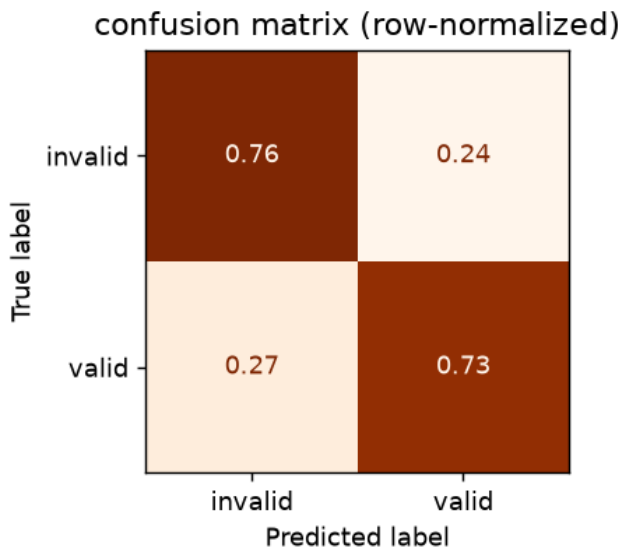
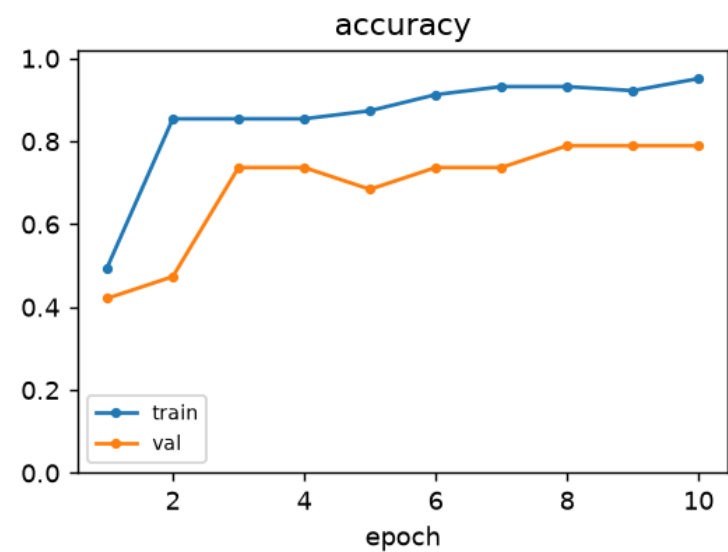
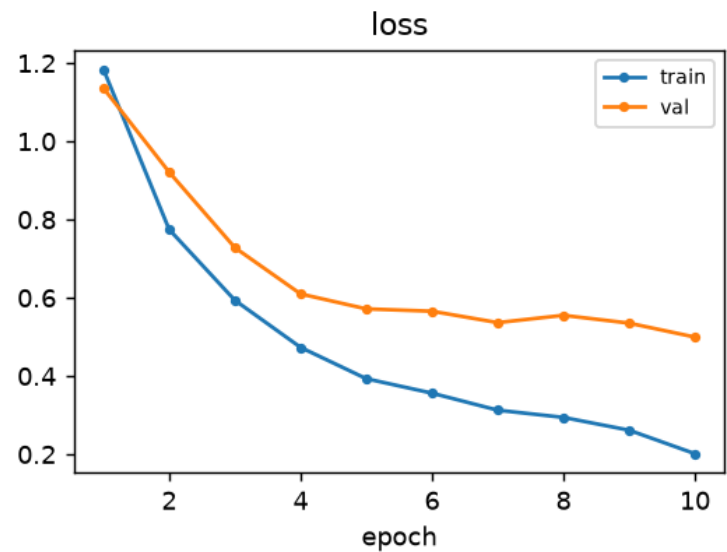
split_75_25 — ratio split



No-empty split (4-class), 70/30 split — promoted model

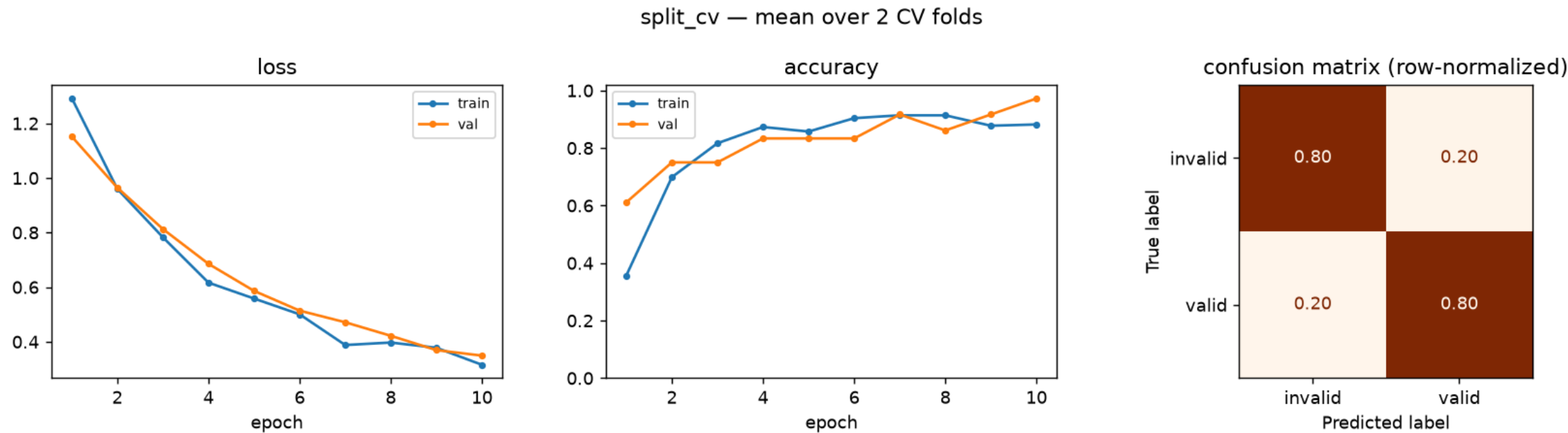
accuracy 74.1% · invalid recall 76.5% · macro F1 0.722 · ROC AUC 0.855 · F1-optimal threshold 0.36 (54 test images)

split_70_30 — ratio split



No-empty split (4-class), 2-fold cross-validation

accuracy 80.3% $\pm$ 0.4 · invalid recall 80.7% $\pm$ 1.6 · macro F1 0.791 · ROC AUC 0.882 · F1-optimal threshold 0.37 (176 pooled test images)



Thank you

Questions?

Code: github.com/yaronconroy-del/picture-recognition-gypsum